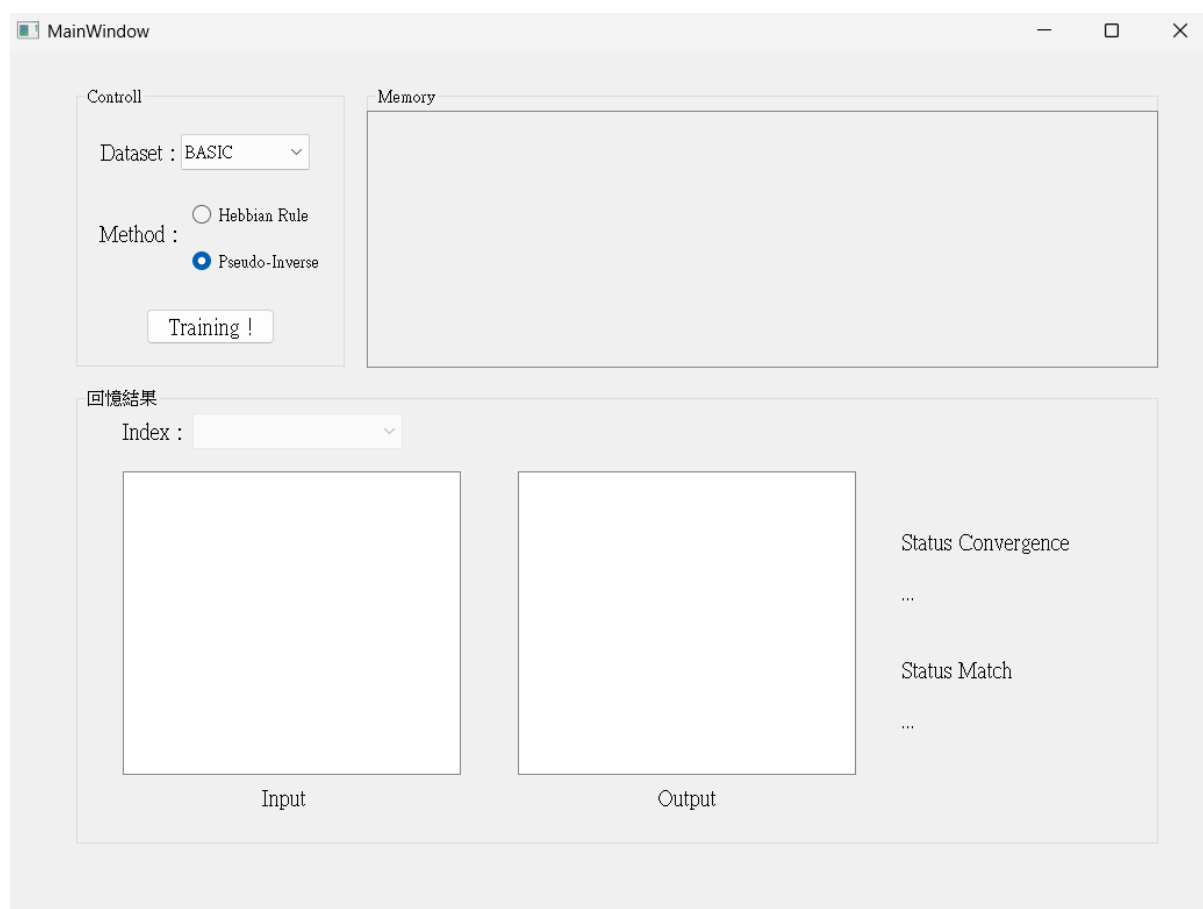


類神經網路作業三 — Hopfield 聯想記憶

A. GUI 功能說明



訓練與模型設定區 (Control)：

此區域用於配置網路的訓練參數

Dataset：選擇訓練資料 (BASIC:基本題，尺寸為 9×12 / BONUS:加分題，尺寸為 10×10)

Method：提供 Hebbian rule 和 Pseudo-inverse 兩種訓練方法

Hebbian rule：使用基礎的赫布規則訓練權重矩陣 W ，適合模式數量較少的場景

Pseudo-Inverse：使用偽逆法則訓練權重矩陣，能夠儲存更多模式並消除模式間的干擾，穩定性更強

Training：根據上述設定載入對應數據，載入數據並計算網路的權重矩陣 W ；訓練完成後將更新記憶展示區和輸入/回想對比

記憶展示區 (Memory)：

此區域以視覺化方式展示網路成功儲存的所有記憶模式

回憶結果區 (回憶結果)：

此區域用於執行聯想回憶測試，並即時顯示網路的收斂狀態

Index：輸入圖片編號選擇要作為 Input 輸入到網路中的測試圖形（來自 Testing.txt 檔案）

Input：展示輸入網路的初始測試圖形

Output：展示 Hopfield 網路經過異步更新迭代後收斂的最終穩定狀態

Status Convergence：顯示網路從初始狀態收斂到最終穩定狀態所經歷的迭代次數

Status Match：驗證最終輸出狀態是否與訓練記憶模式完全吻合

B. 程式碼簡介

1. data_utils.py : 負責資料準備和預處理

bipolarize : 執行 $2X - 1$ 轉換, 將 $0 \rightarrow -1, 1 \rightarrow +1$

unipolarize : 執行 $(X + 1) / 2$ 轉換, 將 $-1 \rightarrow 0, +1 \rightarrow 1$

```
## 1. 數據轉換
# 將 (0, 1) 轉換為 (-1, +1)
def bipolarize(patterns):
    return 2 * patterns - 1

# 將 (-1, +1) 轉換回 (0, 1)
def unipolarize(bipolar_pattern):
    return (bipolar_pattern + 1) // 2
```

load_data : 將每個模式及其後的分隔行視為一個整體(例如 $10 + 1 = 11$ 列), 然後使用 numpy 準確移除最後一行, 確保只有有效數據進入網路

```
## 2. 數據載入
# 讀取 ROWS 行數據, 並忽略最後的單行分隔符
def load_data(file_path, rows, cols):
    # 實際讀取的行數: ROWS (有效數據行) + 1 (單行分隔符)
    READ_ROWS = rows + 1
    TOTAL_READ_SIZE = READ_ROWS * cols

    patterns_list = []
    current_pattern_data = []

    if not os.path.exists(file_path):
        # 錯誤處理將在主程式碼中處理
        return None

    try:
        with open(file_path, 'r', encoding='utf-8') as f:
            lines = f.readlines()

            for line_num, line in enumerate(lines, 1):
                # 清洗輸入行: 處理 Unicode 和不可見字符
                line = line.replace('\xa0', ' ').replace('\t', ' ')
                line_cleaned = ''.join(filter(lambda x: x in string.printable, line))
                line_stripped = line_cleaned.strip()
                is_empty_data_line = not line_stripped

                row_data = [0] * cols

                if not is_empty_data_line:
                    # 逐字符解析前 COLS 個字符
                    for i in range(cols):
                        try:
                            char = line_cleaned[i]
                            if char == '1':
                                row_data[i] = 1
                        except IndexError:
                            break
```

```
# 累積數據點
current_pattern_data.extend(row_data)

# 檢查模式是否完成 (總點數達到 TOTAL_READ_SIZE)
if len(current_pattern_data) == TOTAL_READ_SIZE:
    # 忽略最後 cols 個點 (分隔符)
    valid_pattern_data = np.array(current_pattern_data)[:rows*
patterns_list.append(valid_pattern_data)

    current_pattern_data = [] # 清空

if not patterns_list:
    return None

# 返回 N x M 矩陣 (N: 神經元數, M: 模式數)
return np.stack(patterns_list).T

except Exception as e:
    # 返回 None 讓主程式碼處理
    print(f"錯誤: {e}")
    return None
```

2. `hopfield_core.py` : 負責 hopfield network 的 3 個核心功能：訓練(計算權重矩陣)、回憶(狀態迭代)和驗證(穩定性檢查)，讓前端可以靈活使用不同的學習規則進行實驗比較

`train_hopfield_hebb` : 遵循 Hebbian 學習原則，將每個記憶模式 $S^{(p)}$ 的外部積($S^{(p)}S^{(p)T}$)疊加起來，同時強制設置對角線元素 $W_{ii} = 0$ ，防止神經元自我連結

```
# 使用 Hebbian Rule 計算權重矩陣 W
def train_hopfield_hebb(bipolar_patterns):
    N = bipolar_patterns.shape[0]
    M = bipolar_patterns.shape[1]

    W = np.zeros((N, N))
    for p in range(M):
        s_p = bipolar_patterns[:, p].reshape(-1, 1)
        W += s_p @ s_p.T

    # 設置對角線元素為0 (Wii = 0)
    np.fill_diagonal(W, 0)
    return W
```

`train_hopfield_pseudoinverse` : 使用線性代數中的偽逆運算 S^+ 來計算權重 (即 $W = SS^+$)，刻意允許 $W_{ii} \neq 0$ ，這有助於在偽逆法則下進一步強化穩定性並加速收斂

```
# 使用 Pseudo-Inverse Rule 計算權重矩陣 W
def train_hopfield_pseudoinverse(bipolar_patterns):
    N = bipolar_patterns.shape[0]
    # M = bipolar_patterns.shape[1]

    S = bipolar_patterns

    try:
        # 計算 S 的偽逆 S_pinv (M x N 矩陣)
        S_pinv = pinv(S)
    except np.linalg.LinAlgError as e:
        print(f"pseudo-inverse計算失敗:{e}")
        return np.zeros((N, N))

    # 計算 W = S @ S_pinv (N x N 矩陣)
    W = S @ S_pinv

    # 設置對角線元素為0 (Wii = 0) - *在偽逆法則中，允許 Wii != 0 通常效果更好*
    # np.fill_diagonal(W, 0)
    return W
```

recall_pattern：模擬 hopfield network 在給定初始狀態下的動態行為，並採用異步更新策略(在每一輪疊代中隨機選擇神經元進行更新)，確保系統穩定性和避免震盪。網路的狀態會持續疊代更新直到當前狀態與前一狀態完全相同，輸出包含最終收斂的狀態和收斂所需的迭代輪數

```
## 2. 回憶階段：狀態迭代收斂
# 使用異步更新來回憶模式，直到收斂
def recall_pattern(W, initial_state, max_iter=1000):
    N = W.shape[0]
    current_state = initial_state.copy()

    # 異步更新
    for iteration in range(max_iter):
        previous_state = current_state.copy()
        update_order = list(range(N))
        random.shuffle(update_order)

        for i in update_order:
            # 計算淨輸入
            net_input = W[i, :] @ current_state

            # 激活函數：sign(h_i)
            if net_input > 0:
                current_state[i] = 1
            elif net_input < 0:
                current_state[i] = -1
            # 否則 (net_input == 0) 狀態保持不變

        if np.array_equal(current_state, previous_state):
            print(f"收斂於第{iteration + 1}輪迭代")
            return current_state, iteration + 1

    return current_state, max_iter
```

verify_spurious_state：將最終收斂的狀態與所有訓練記憶進行逐一比對，判斷網路是否收斂到預期的訓練模式，如果最終狀態和任何訓練記憶都不匹配，則判定為收斂到一個虛假記憶

```
## 3. 驗證：虛偽記憶檢查
# 檢查最終狀態是否為已儲存的真實記憶
def verify_spurious_state(final_state, training_bipolar):
    M = training_bipolar.shape[1]

    for j in range(M):
        memory_Pj = training_bipolar[:, j]

        # 檢查輸出是否與任一記憶模式完全相同
        if np.array_equal(final_state, memory_Pj):
            return True, j

    # 如果迴圈結束都未匹配，則視為偽記憶
    return False, -1
```

3. main.py：扮演 Qt（前端）和 hopfield_core（後端）演算法之間的橋樑

HopfieldGUIConnector：此類別是一個狀態容器，負責儲存網路的關鍵變數（如權重矩陣、訓練模式、當前圖形尺寸等），確保網路完成訓練後，狀態能夠被回憶功能持續使用

```
# 負責處理所有數據載入、訓練和回憶邏輯，作為 Qt 介面的後端接口
class HopfieldGUIConnector:
    def __init__(self):
        self.W = None
        self.training_bipolar = None
        self.testing_bipolar = None
        self.rows = 0
        self.cols = 0
        self.current_dataset = None
```

load_and_train：從 GUI 接收參數並完成整個網路的訓練流程

```
# 載入數據並訓練網路
def load_and_train(self, dataset_key, rule):
    config = DATA_CONFIG.get(dataset_key.upper())
    if not config:
        raise ValueError("無效的數據集選擇。")

    self.current_dataset = dataset_key.upper()
    self.rows = config['ROWS']
    self.cols = config['COLS']

    # 1. 載入訓練數據
    print(f"載入 {self.rows}x{self.cols} 訓練數據...")
    training_uni = utils.load_data(config['TRAIN_FILE'], self.rows, self.cols)
    if training_uni is None:
        raise FileNotFoundError(f"無法載入訓練檔案: {config['TRAIN_FILE']}")

    self.training_bipolar = utils.bipolarize(training_uni)
    print(f"成功載入 {self.training_bipolar.shape[1]} 個訓練模式。")

    # 2. 訓練：計算權重矩陣 W
    self.W = core.get_trained_weights(self.training_bipolar, rule)
    print(f"權重矩陣 W 計算完成 (大小: {self.W.shape[0]} x {self.W.shape[1]})")

    # 3. 載入測試數據
    testing_uni = utils.load_data(config['TEST_FILE'], self.rows, self.cols)
    if testing_uni is not None:
        self.testing_bipolar = utils.bipolarize(testing_uni)
    else:
        self.testing_bipolar = None

    # 返回訓練模式數量和測試模式數量，供 GUI 更新下拉選單
    num_train = self.training_bipolar.shape[1]
    num_test = self.testing_bipolar.shape[1] if self.testing_bipolar is not None else 0

    return self.training_bipolar, num_train, num_test
```

recall_test_pattern：模擬單次回憶測試，並將結果回傳給 GUI 顯示

```
# 根據測試索引進行單次回憶
def recall_test_pattern(self, test_index, noise_percent=0):
    if self.W is None:
        raise RuntimeError("請先訓練網路。")
    if self.testing_bipolar is None:
        raise RuntimeError("未載入測試資料。")

    # 檢查索引是否有效
    if test_index < 0 or test_index >= self.testing_bipolar.shape[1]:
        raise IndexError("測試模式索引超出範圍。")

    # 1. 從測試集取得初始輸入
    # 由於暫無 add_noise，我們直接使用測試集中的模式作為輸入
    initial_input = self.testing_bipolar[:, test_index].copy()
    noisy_input = initial_input # 0% 雜訊

    # 2. 回憶
    final_state, convergence_iter = core.recall_pattern(self.W, noisy_input)

    # 3. 驗證
    is_memory, matched_index = core.verify_spurious_state(final_state, self.W)

    # 返回結果，供 GUI 繪製
    return noisy_input, final_state, is_memory, matched_index, convergence_iter
```


4. hopfield_gui.py : 負責將複雜的矩陣運算結果轉為直觀的圖形和狀態信息

handle_train_click : 從 GUI 收集 user 的選擇，並協調後端 HopfieldGUIConnector 執行訓練

```
# 點擊 '開始訓練' 按鈕時執行
def handle_train_click(self):
    try:
        # 1. 讀取用戶選擇
        dataset_key = self.comboBox_dataset.currentText()

        # 判斷訓練規則
        if self.ui.radio_hebb.isChecked():
            rule = "HEBB"
        elif self.ui.radio_pinv.isChecked():
            rule = "PINV"
        else:
            QMessageBox.warning(self, "錯誤", "請選擇訓練規則 (Hebb Rule)")
            return

        # 2. 調用後端接口進行載入和訓練
        train_bipolar, num_train, num_test = self.connector.load_and_train(
            dataset_key, rule, self.ui.spinBox_patterns.value(),
            self.ui.spinBox_rows.value(), self.ui.spinBox_cols.value(),
            self.ui.spinBox_epochs.value(), self.ui.spinBox_test_size.value())

        self.current_patterns_count = num_train
        self.current_rows = self.connector.rows
        self.current_cols = self.connector.cols

        # 3. 更新記憶展示區
        self.display_memories(train_bipolar)

        # 4. 更新回憶測試下拉選單
        self.update_test_selector(num_test)

        QMessageBox.information(self, "訓練成功", f"網路已訓練完成 ({rule})")

    except Exception as e:
        QMessageBox.critical(self, "訓練錯誤", f"訓練或載入數據失敗:\n{e}")
```

display_memories : 專門處理記憶 memory 的數據，確保所有訓練模式都能清晰、有編號地被展示出來

```
# 在 widget_memories 中繪製所有記憶模式 (使用QLabel + QPixmap)
def display_memories(self, train_bipolar):
    if not hasattr(self, 'widget_memories'):
        return

    # 清除舊的佈局和內容
    if self.widget_memories.layout():
        QWidget().setLayout(self.widget_memories.layout())

    M = train_bipolar.shape[1]

    # 設置尺寸參數 (這是給 QLabel 設置的最小尺寸)
    MEM_SIZE = 100 # 讓每個圖案顯示為 100x100 像素
    PLOT_COLS = min(M, 5)
```

```

grid_layout = QGridLayout(self.widget_memories)
grid_layout.setSpacing(5)

for i in range(M):
    pattern_vector = train_bipolar[:, i]

    # 創建 QPixmap 圖像
    pixmap = create_pixmap_from_pattern(
        pattern_vector, self.current_rows, self.current_cols, size
    )

    # 創建 QLabel 來顯示圖像
    img_label = QLabel(self.widget_memories)
    img_label.setPixmap(pixmap)
    img_label.setAlignment(Qt.AlignCenter) # 圖像居中
    img_label.setFixedSize(QSize(MEM_SIZE, MEM_SIZE)) # 鎖定 QLabel

    # 創建編號 Label
    label = QLabel(f"#{i + 1}")
    label.setAlignment(Qt.AlignCenter)

    # 創建一個垂直佈局 VBox
    vbox = QVBoxLayout()
    vbox.addWidget(label)
    vbox.addWidget(img_label) # 顯示圖像的 Label

    container = QWidget(self.widget_memories)
    container.setLayout(vbox)

    row = i // PLOT_COLS
    col = i % PLOT_COLS
    grid_layout.addWidget(container, row, col)

self.widget_memories.setLayout(grid_layout)

```

handle_recall_test：在 user 選擇測試模式時觸發，執行單次聯想回憶並顯示詳細的結果和狀態

```

# 當選擇測試模式時執行回憶 (接收 currentIndexChanged 傳來的 index)
def handle_recall_test(self, index):
    if self.connector.W is None:
        return # 尚未訓練

    # 直接使用傳入的 index
    test_index = index
    if test_index == -1:
        return # 下拉選單為空

    try:
        # 暫時將 noise_percent 設為 0
        noisy_input, final_state, is_memory, matched_index, convergence = self.run_test(
            test_index=test_index,
            noise_percent=0
        )

        # 1. 顯示輸入圖形
        self.display_pattern_on_view(self.graphicsView_input, noisy_in

        # 2. 顯示回憶結果
        self.display_pattern_on_view(self.graphicsView_output, final_s

```

```
# 3. 更新回憶結果訊息
if convergence_iter < 1000:
    self.label_status_convergence.setText(f"✅ 收斂於第 {convergence_iter}")
else:
    self.label_status_convergence.setText("未收斂 (Max Iter)")

if is_memory:
    self.label_status_match.setText(f"✅ 訓練記憶 #{matched_in}")
else:
    self.label_status_match.setText("❌ 虛偽/錯誤收斂")

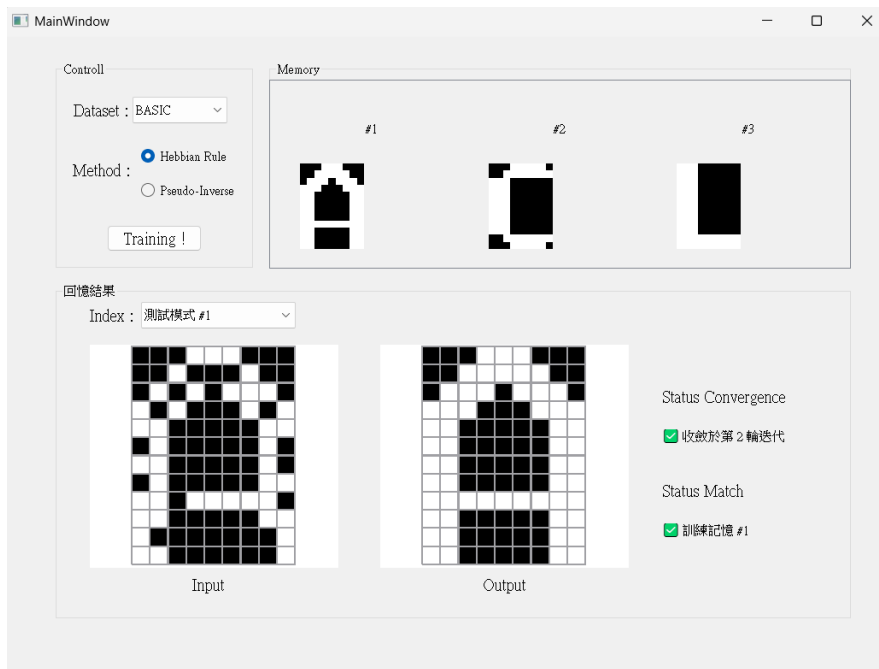
except Exception as e:
    QMessageBox.critical(self, "回憶錯誤", f"回憶測試失敗：\n{e}")
```

C. 實驗結果展示

(本部分只記錄四種參數配置下觀察到的”部分”客觀結果)

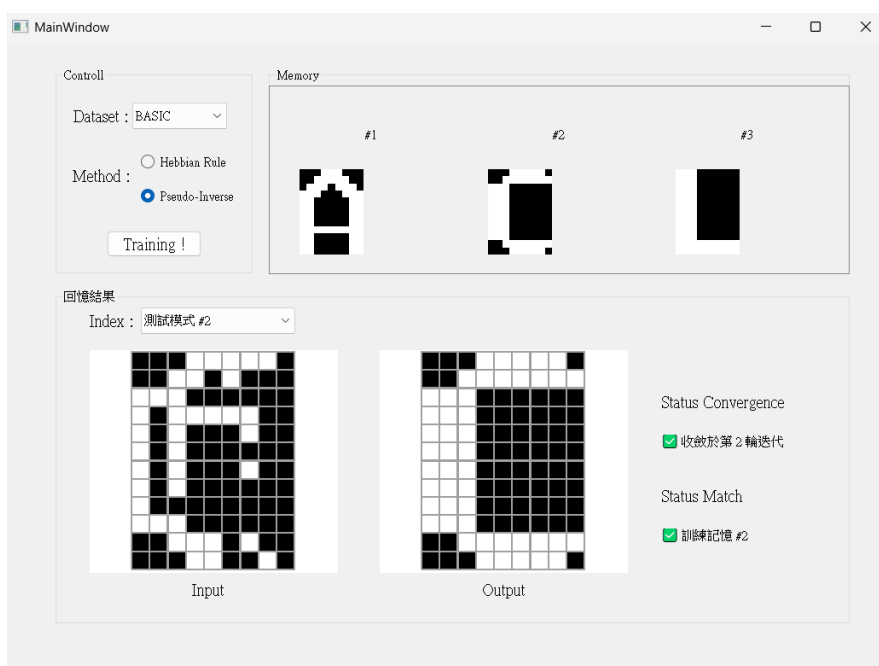
BASIC X Hebb (網路成功儲存 $M = 3$ 個模式)

測試模式#1 經回憶收斂到訓練記憶#1 / 收斂輪數：2 輪



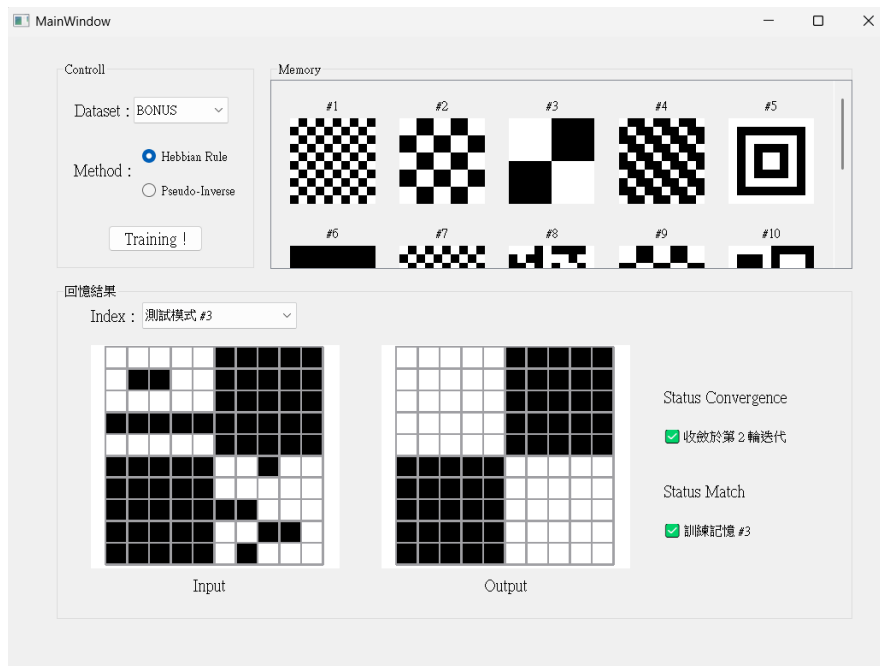
BASIC X Pseudo-Inverse (網路成功儲存 $M = 3$ 個模式)

測試模式#2 成功收斂到訓練記憶#2 / 收斂輪數：2 輪



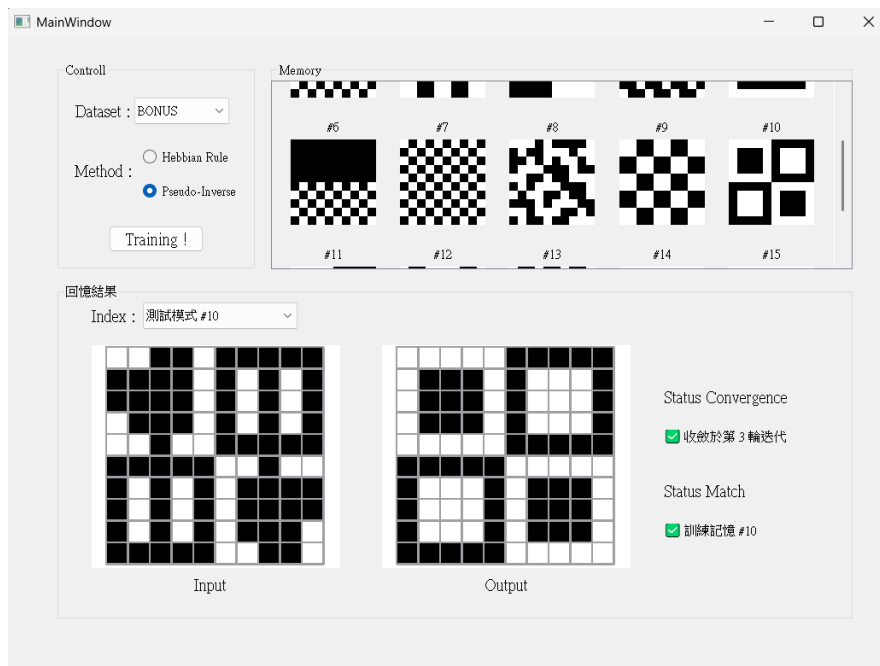
BONUS X Hebb (網路成功儲存 $M = 15$ 個模式)

測試模式#3 成功收斂到訓練記憶#3 / 收斂輪數：2 輪



BONUS X Pseudo-Inverse (網路成功儲存 $M = 15$ 個模式)

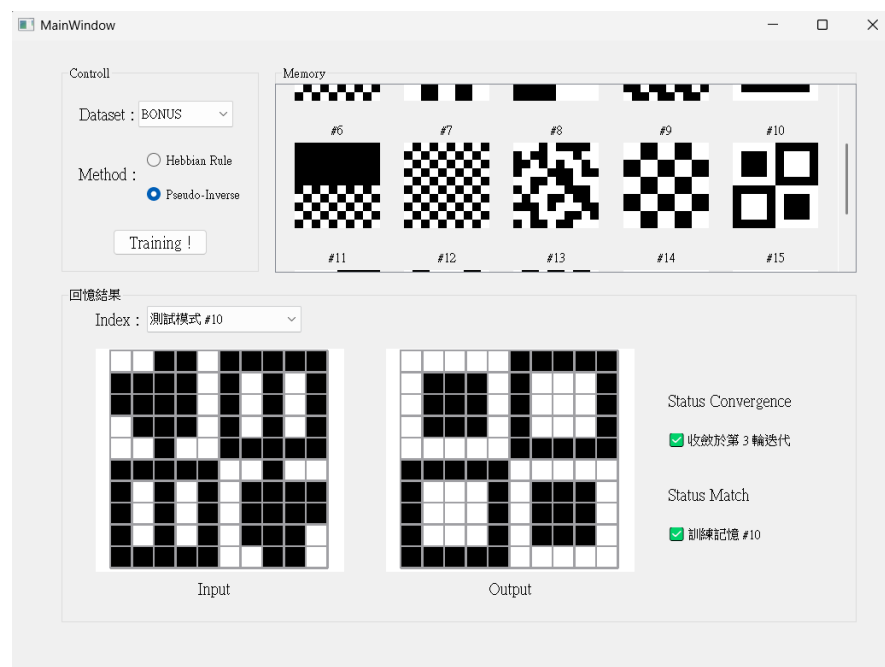
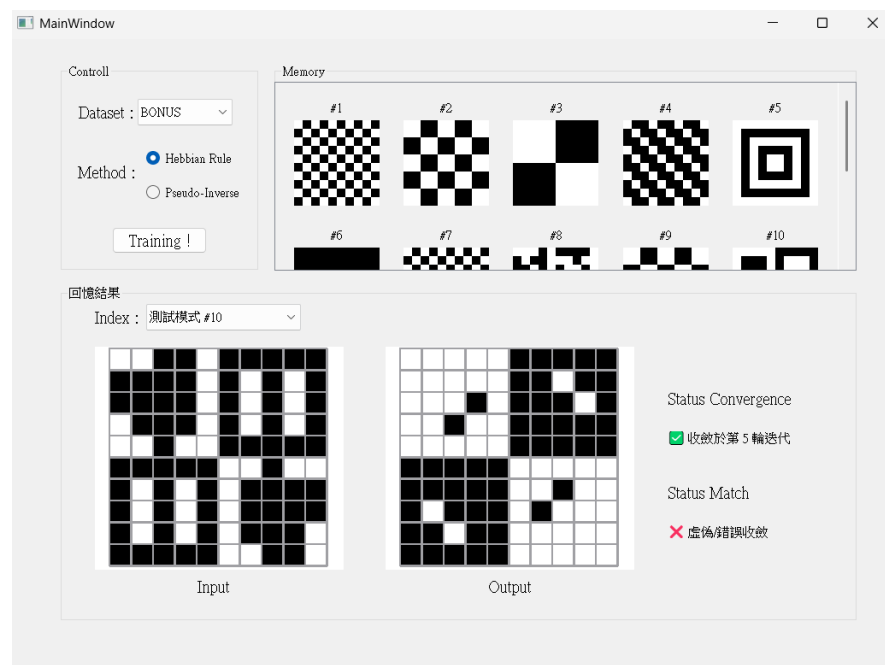
測試模式#10 成功收斂到訓練記憶#10 / 收斂輪數：3 輪。



D. 結果分析

測試模式#10：網路收斂於第5輪迭代，但匹配記憶為虛假記憶

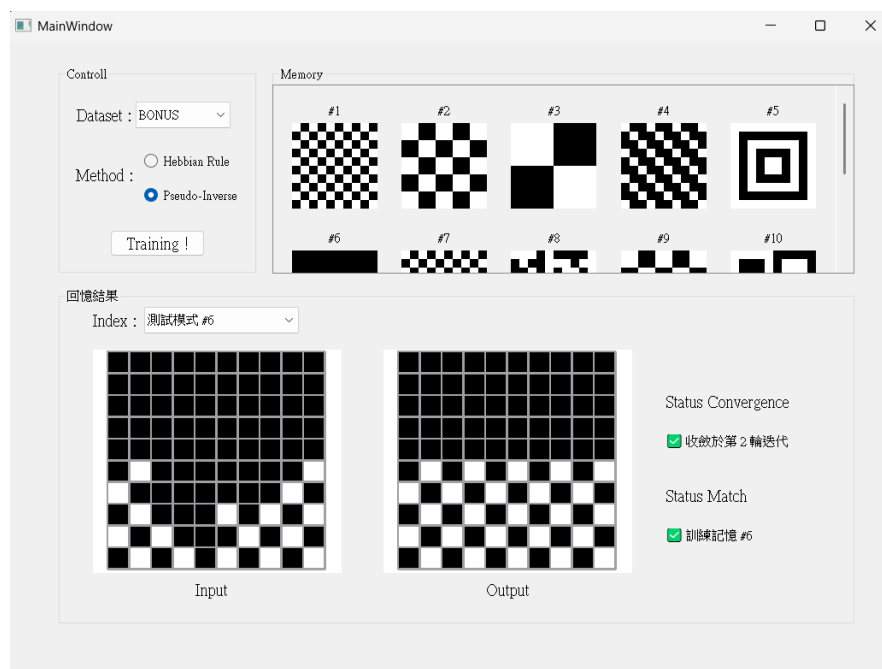
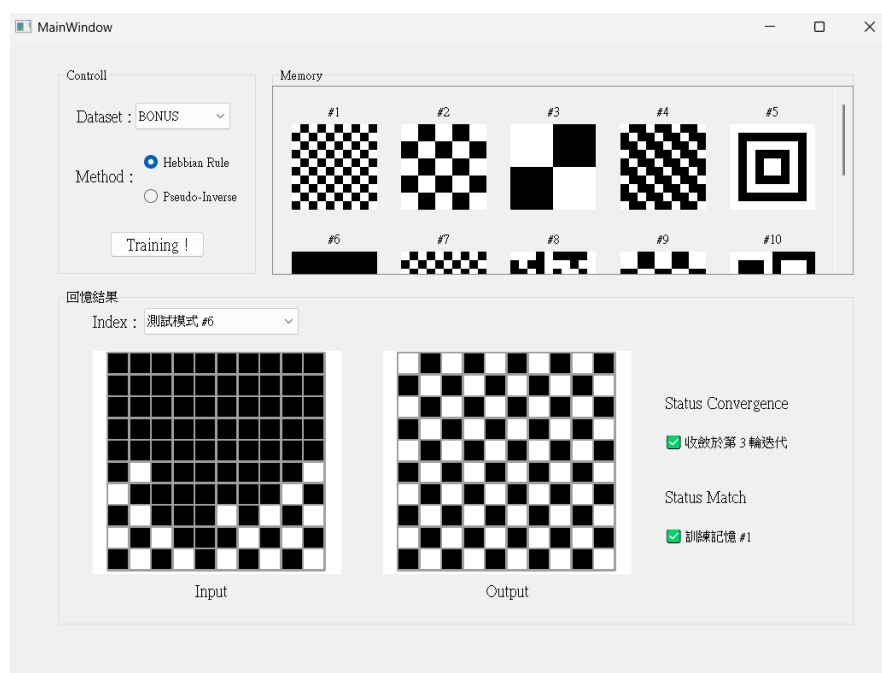
這表明在模式數量超過容量限制時，網路的能量盆地被扭曲，導致輸入狀態被吸引到記憶模式之間的非訓練穩定點。



比較：當模式數量過多，Hebb 的權重矩陣會累積大量的模式間干擾項，導致測試輸入被拉進非穩定的訓練點；相較之下，pseudo-inverse 計算出的權重矩陣能消除所有模式間的線性依賴項，有效地去除了 Hebb 中的干擾項，因此能成功且穩定收斂到訓練記憶。

測試模式#6：網路收斂於第3輪迭代，成功匹配訓練記憶#1

從視覺上看，輸出圖像雖然是訓練記憶，但與輸入圖形的相似度似乎不如預期（因為這張圖明顯跟訓練記憶#6 比較相似）。



比較：即使 Hebb 成功回憶，我仍主觀認為回憶結果不理想，這也是干擾項的另一個體現，因為干擾項使得穩定點的邊界模糊，網路雖然收斂到正確的記憶，但是收斂品質較差；相較之下，pseudo-inverse 收斂到的記憶看起來就合理許多(收斂品質更好)。

藉由以上例子的比較，可觀察出 pseudo-inverse 具有較高的容量和穩定性。